Importing Modules

When you first start up a Python interpreter (or a new Jupyter notebook), there is only a limited set of commands and objects available for you to work with. To extend this functionality, there is a vast collection of tools that extend the functionality of Python, but that must be explicitly *imported* to make them available.

This is in contrast to tools like MATLAB, Igor, and R, which implicitly provide access to a larger array of functionality. This is a reflection of the fact that Python is a general-purpose language; whereas MATLAB assumes the user is interested in linear algebra, Python tries to make no assumptions about what functionality we will need.

Libraries

Python itself comes with a large number of handy standard libraries that can be imported and used:

Python Standard Library

However, the real power of Python for *scientific computing* comes from the wealth of open-source, community driven packages that are available. Many of these external packages come bundled with the Anaconda Python distribution.

To help prepare you for the Summer Workshop on the Dynamic Brain, we are going to focus on several popular and powerful scientific computing libraries:

- Numpy - Efficient numerical arrays, linear algebra, and optimized mathematical functions (http://www.numpy.org)
- Scipy - A suite of powerful scientific modules that extends on Numpy's functionality, providing signal and image processing, statistical tools, optimization, interpolation, and more (http://docs.scipy.org/doc/scipy/reference/)
- Matplotlib - A 2D plotting library that produces publication quality figures (http://matplotlib.org/)
- Scikit-learn - Easy to use machine learning (https://scikit-learn.org)
- Pandas - A library for manipulation of tabular, relational data (http://pandas.pydata.org/)

Packages and Modules

Before we get into the details of these packages, we need some definitions:

Module:

1. A file that contains the code defining a set of python functions, classes, and data.
2. A type of Python object that contains a set of python functions, classes, and data. Modules are loaded by Python using the `import` statement, and the functionality contained in the module is accessed as attributes of the module object.

Package:

1. A special type of module (object) that contains submodules and/or subpackages, providing a hierarchical organization for modules. Packages, just like modules, are loaded using the `import` statement.
2. A container of one or more related Python modules that can be installed together (for example, numpy is a package containing many submodules. It can be downloaded, installed, and imported.) The name "library" is sometimes used interchangeably with package.

Let's start by importing numpy. This is one of the most important libraries in the scientific Python stack; we will explore it in detail in the next section.

In [1]:
```
import numpy
```

This statement did two things:

1. Created a new module object by loading the "numpy" module
2. Assigned the name "numpy" to this new object

In [2]:
```
numpy
```

Out[2]:
```
<module 'numpy' from '/home/luke/miniconda3/envs/swdb/lib/python3.12/site-packages/numpy/__init__.py'>
```

The functionality contained within this module are accessed as *attributes*:

In [3]:
```
numpy.pi
```

Out[3]:
```
3.141592653589793
```

Numpy is a large library. Let's use the `dir` function, which you learned earlier, to view all of the available methods.

In [4]:
```
print(dir(numpy))
```

['ALLOW_THREADS', 'BUFSIZE', 'CLIP', 'DataSource', 'ERR_CALL', 'ERR_DEFAUL
T', 'ERR_IGNORE', 'ERR_LOG', 'ERR_PRINT', 'ERR_RAISE', 'ERR_WARN', 'FLOATING
_POINT_SUPPORT', 'FPE_DIVIDEBYZERO', 'FPE_INVALID', 'FPE_OVERFLOW', 'FPE_UND
ERFLOW', 'False_', 'Inf', 'Infinity', 'MAXDIMS', 'MAY_SHARE_BOUNDS', 'MAY_SH
ARE_EXACT', 'NAN', 'NINF', 'NZERO', 'NaN', 'PINF', 'PZERO', 'RAISE', 'RankWa
rning', 'SHIFT_DIVIDEBYZERO', 'SHIFT_INVALID', 'SHIFT_OVERFLOW', 'SHIFT_UNDE
RFLOW', 'ScalarType', 'True_', 'UFUNC_BUFSIZE_DEFAULT', 'UFUNC_PYVALS_NAME',
'WRAP', '_CopyMode', '_NoValue', '_UFUNC_API', '__NUMPY_SETUP__', '__all__',
'__builtins__', '__cached__', '__config__', '__deprecated_attrs__', '__dir_
_', '__doc__', '__expired_functions__', '__file__', '__former_attrs__', '__f
uture_scalars__', '__getattr__', '__loader__', '__name__', '__package__', '_
_path__', '__spec__', '__version__', '_add_newdoc_ufunc', '_builtins', '_dis
tributor_init', '_financial_names', '_get_promotion_state', '_globals', '_in
t_extended_msg', '_mat', '_no_nep50_warning', '_pyinstaller_hooks_dir', '_py
testtester', '_set_promotion_state', '_specific_msg', '_typing', '_using_num
py2_behavior', '_utils', 'abs', 'absolute', 'add', 'add_docstring', 'add_new
doc', 'add_newdoc_ufunc', 'all', 'allclose', 'alltrue', 'amax', 'amin', 'ang
le', 'any', 'append', 'apply_along_axis', 'apply_over_axes', 'arange', 'arcc
os', 'arccosh', 'arcsin', 'arcsinh', 'arctan', 'arctan2', 'arctanh', 'argma
x', 'argmin', 'argpartition', 'argsort', 'argwhere', 'around', 'array', 'arr
ay2string', 'array_equal', 'array_equiv', 'array_repr', 'array_split', 'arra
y_str', 'asanyarray', 'asarray', 'asarray_chkfinite', 'ascontiguousarray',
'asfarray', 'asfortranarray', 'asmatrix', 'atleast_1d', 'atleast_2d', 'atlea
st_3d', 'average', 'bartlett', 'base_repr', 'binary_repr', 'bincount', 'bitw
ise_and', 'bitwise_not', 'bitwise_or', 'bitwise_xor', 'blackman', 'block',
'bmat', 'bool_', 'broadcast', 'broadcast_arrays', 'broadcast_shapes', 'broad
cast_to', 'busday_count', 'busday_offset', 'busdaycalendar', 'byte', 'byte_b
ounds', 'bytes_', 'c_', 'can_cast', 'cast', 'cbrt', 'cdouble', 'ceil', 'cflo
at', 'char', 'character', 'chararray', 'choose', 'clip', 'clongdouble', 'clo
ngfloat', 'column_stack', 'common_type', 'compare_chararrays', 'compat', 'co
mplex128', 'complex256', 'complex64', 'complex_', 'complexfloating', 'compre
ss', 'concatenate', 'conj', 'conjugate', 'convolve', 'copy', 'copysign', 'co
pyto', 'corrcoef', 'correlate', 'cos', 'cosh', 'count_nonzero', 'cov', 'cros
s', 'csingle', 'ctypeslib', 'cumprod', 'cumproduct', 'cumsum', 'datetime64',
'datetime_as_string', 'datetime_data', 'deg2rad', 'degrees', 'delete', 'depr
ecate', 'deprecate_with_doc', 'diag', 'diag_indices', 'diag_indices_from',
'diagflat', 'diagonal', 'diff', 'digitize', 'disp', 'divide', 'divmod', 'do
t', 'double', 'dsplit', 'dstack', 'dtype', 'dtypes', 'e', 'ediff1d', 'einsu
m', 'einsum_path', 'emath', 'empty', 'empty_like', 'equal', 'errstate', 'eul
er_gamma', 'exceptions', 'exp', 'exp2', 'expand_dims', 'expm1', 'extract',
'eye', 'fabs', 'fastCopyAndTranspose', 'fft', 'fill_diagonal', 'find_common_
type', 'finfo', 'fix', 'flatiter', 'flatnonzero', 'flexible', 'flip', 'flipl
r', 'flipud', 'float128', 'float16', 'float32', 'float64', 'float_', 'float_
power', 'floating', 'floor', 'floor_divide', 'fmax', 'fmin', 'fmod', 'format
_float_positional', 'format_float_scientific', 'format_parser', 'frexp', 'fr
om_dlpack', 'frombuffer', 'fromfile', 'fromfunction', 'fromiter', 'frompyfun
c', 'fromregex', 'fromstring', 'full', 'full_like', 'gcd', 'generic', 'genfr
omtxt', 'geomspace', 'get_array_wrap', 'get_include', 'get_printoptions', 'g
etbufsize', 'geterr', 'geterrcall', 'geterrobj', 'gradient', 'greater', 'gre
ater_equal', 'half', 'hamming', 'hanning', 'heaviside', 'histogram', 'histog
ram2d', 'histogram_bin_edges', 'histogramdd', 'hsplit', 'hstack', 'hypot',
'i0', 'identity', 'iinfo', 'imag', 'in1d', 'index_exp', 'indices', 'inexac
t', 'inf', 'info', 'infty', 'inner', 'insert', 'int16', 'int32', 'int64', 'i
nt8', 'int_', 'intc', 'integer', 'interp', 'intersect1d', 'intp', 'invert',
'is_busday', 'isclose', 'iscomplex', 'iscomplexobj', 'isfinite', 'isfortra
n', 'isin', 'isinf', 'isnan', 'isnat', 'isneginf', 'isposinf', 'isreal', 'is

```
realobj', 'isscalar', 'issctype', 'issubclass_', 'issubdtype', 'issubsctyp
e', 'iterable', 'ix_', 'kaiser', 'kernel_version', 'kron', 'lcm', 'ldexp',
'left_shift', 'less', 'less_equal', 'lexsort', 'lib', 'linalg', 'linspace',
'little_endian', 'load', 'loadtxt', 'log', 'log10', 'log1p', 'log2', 'logadd
exp', 'logaddexp2', 'logical_and', 'logical_not', 'logical_or', 'logical_xo
r', 'logspace', 'longcomplex', 'longdouble', 'longfloat', 'longlong', 'lookf
or', 'ma', 'mask_indices', 'mat', 'matmul', 'matrix', 'max', 'maximum', 'max
imum_sctype', 'may_share_memory', 'mean', 'median', 'memmap', 'meshgrid', 'm
grid', 'min', 'min_scalar_type', 'minimum', 'mintypecode', 'mod', 'modf', 'm
oveaxis', 'msort', 'multiply', 'nan', 'nan_to_num', 'nanargmax', 'nanargmi
n', 'nancumprod', 'nancumsum', 'nanmax', 'nanmean', 'nanmedian', 'nanmin',
'nanpercentile', 'nanprod', 'nanquantile', 'nanstd', 'nansum', 'nanvar', 'nb
ytes', 'ndarray', 'ndenumerate', 'ndim', 'ndindex', 'nditer', 'negative', 'n
ested_iters', 'newaxis', 'nextafter', 'nonzero', 'not_equal', 'numarray', 'n
umber', 'obj2sctype', 'object_', 'ogrid', 'oldnumeric', 'ones', 'ones_like',
'outer', 'packbits', 'pad', 'partition', 'percentile', 'pi', 'piecewise', 'p
lace', 'poly', 'poly1d', 'polyadd', 'polyder', 'polydiv', 'polyfit', 'polyin
t', 'polymul', 'polynomial', 'polysub', 'polyval', 'positive', 'power', 'pri
ntoptions', 'prod', 'product', 'promote_types', 'ptp', 'put', 'put_along_axi
s', 'putmask', 'quantile', 'r_', 'rad2deg', 'radians', 'random', 'ravel', 'r
avel_multi_index', 'real', 'real_if_close', 'rec', 'recarray', 'recfromcsv',
'recfromtxt', 'reciprocal', 'record', 'remainder', 'repeat', 'require', 'res
hape', 'resize', 'result_type', 'right_shift', 'rint', 'roll', 'rollaxis',
'roots', 'rot90', 'round', 'round_', 'row_stack', 's_', 'safe_eval', 'save',
'savetxt', 'savez', 'savez_compressed', 'sctype2char', 'sctypeDict', 'sctype
s', 'searchsorted', 'select', 'set_numeric_ops', 'set_printoptions', 'set_st
ring_function', 'setbufsize', 'setdiff1d', 'seterr', 'seterrcall', 'seterrob
j', 'setxor1d', 'shape', 'shares_memory', 'short', 'show_config', 'show_runt
ime', 'sign', 'signbit', 'signedinteger', 'sin', 'sinc', 'single', 'singleco
mplex', 'sinh', 'size', 'sometrue', 'sort', 'sort_complex', 'source', 'spaci
ng', 'split', 'sqrt', 'square', 'squeeze', 'stack', 'std', 'str_', 'string
_', 'subtract', 'sum', 'swapaxes', 'take', 'take_along_axis', 'tan', 'tanh',
'tensordot', 'test', 'testing', 'tile', 'timedelta64', 'trace', 'tracemalloc
_domain', 'transpose', 'trapz', 'tri', 'tril', 'tril_indices', 'tril_indices
_from', 'trim_zeros', 'triu', 'triu_indices', 'triu_indices_from', 'true_div
ide', 'trunc', 'typecodes', 'typename', 'ubyte', 'ufunc', 'uint', 'uint16',
'uint32', 'uint64', 'uint8', 'uintc', 'uintp', 'ulonglong', 'unicode_', 'uni
on1d', 'unique', 'unpackbits', 'unravel_index', 'unsignedinteger', 'unwrap',
'ushort', 'vander', 'var', 'vdot', 'vectorize', 'version', 'void', 'vsplit',
'vstack', 'where', 'who', 'zeros', 'zeros_like']
```

> We can also use the `dir` command without an argument to see what's in our notebook's global 'namespace':

In [5]:
```
print(dir())
```

```
['In', 'Out', '_', '_2', '_3', '__', '___', '__builtin__', '__builtins__',
'__doc__', '__loader__', '__name__', '__package__', '__spec__', '_dh', '_i',
'_i1', '_i2', '_i3', '_i4', '_i5', '_ih', '_ii', '_iii', '_oh', 'exit', 'get
_ipython', 'numpy', 'open', 'quit']
```

> Most of these are behind-the-scenes variables that your kernel is using to keep track of command history or other low-level attributes.

At this point, the only variable we've actually declared is `numpy`

Importing specific functions from a module

Another importing option is to import the specific functions that you need from a libary

In [6]:
```
from numpy import cos,pi
```

Note that, if we now look at the global namespace using `dir`, the two names that we've just imported are now visible, which means that they're globally accessible.

In [7]:
```
print(dir())
```

```
['In', 'Out', '_', '_2', '_3', '__', '___', '__builtin__', '__builtins__',
'__doc__', '__loader__', '__name__', '__package__', '__spec__', '_dh', '_i',
'_i1', '_i2', '_i3', '_i4', '_i5', '_i6', '_i7', '_ih', '_ii', '_iii', '_o
h', 'cos', 'exit', 'get_ipython', 'numpy', 'open', 'pi', 'quit']
```

In [8]:
```
cos(pi)
```

Out[8]:
```
-1.0
```

Importing all attributes into the global namespace - A word of caution:

It is possible to import all objects from a module:

```
    from numpy import *
```

THIS IS GENERALLY DISCOURAGED. It makes a mess of your namespace, it makes it difficult to tell where any given name was defined, and it increases the likelihood of name conflicts. In addition, against intuitive interpretation, it does not "import everything". The behavior of this command is defined by the implementation of the module, so it can be difficult to predict what you will get.

This is a common way to import:

Packages can be renamed on import and standard shorthand notation has developed for various packages. For instance, Numpy is commonly imported as `np`. Following this convention is not required, but doing so makes your code a.

In [9]:
```
import numpy as np
```

In [10]:
```
print(np.sqrt(np.pi))
```

```
1.7724538509055159
```

Installing new modules

Python comes with a large set of useful packages pre-installed, and Anaconda builds on those with an even larger set of computational tools. But every once in a while, we may need to install something new.

The most common place for developers to publish new packages is on the Python Package Index, and the easiest way to install these packages is with the `pip` command:

In [11]:
```
# NOTE: This is not Python!
# Starting a line with ! tells Jupyter to run a shell command instead.
# you could achieve the same thing by running `pip install statsmodels`
# in your conda-activated terminal
!pip install statsmodels
```

```
Requirement already satisfied: statsmodels in /home/luke/miniconda3/envs/swd
b/lib/python3.12/site-packages (0.14.2)
Requirement already satisfied: numpy>=1.22.3 in /home/luke/miniconda3/envs/s
wdb/lib/python3.12/site-packages (from statsmodels) (1.26.4)
Requirement already satisfied: scipy!=1.9.2,>=1.8 in /home/luke/miniconda3/e
nvs/swdb/lib/python3.12/site-packages (from statsmodels) (1.13.0)
Requirement already satisfied: pandas!=2.1.0,>=1.4 in /home/luke/miniconda3/
envs/swdb/lib/python3.12/site-packages (from statsmodels) (2.2.1)
Requirement already satisfied: patsy>=0.5.6 in /home/luke/miniconda3/envs/sw
db/lib/python3.12/site-packages (from statsmodels) (0.5.6)
Requirement already satisfied: packaging>=21.3 in /home/luke/miniconda3/env
s/swdb/lib/python3.12/site-packages (from statsmodels) (23.2)
Requirement already satisfied: python-dateutil>=2.8.2 in /home/luke/minicond
a3/envs/swdb/lib/python3.12/site-packages (from pandas!=2.1.0,>=1.4->statsmo
dels) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /home/luke/miniconda3/envs/sw
db/lib/python3.12/site-packages (from pandas!=2.1.0,>=1.4->statsmodels) (202
4.1)
Requirement already satisfied: tzdata>=2022.7 in /home/luke/miniconda3/envs/
swdb/lib/python3.12/site-packages (from pandas!=2.1.0,>=1.4->statsmodels) (2
023.3)
Requirement already satisfied: six in /home/luke/miniconda3/envs/swdb/lib/py
thon3.12/site-packages (from patsy>=0.5.6->statsmodels) (1.16.0)
```

Note that when we used `pip` to install `statsmodels`, it also ensured that any other packages required by `statsmodels` are installed at the same time.

In [12]:
```
import statsmodels
statsmodels
```

Out[12]:
```
<module 'statsmodels' from '/home/luke/miniconda3/envs/swdb/lib/python3.12/
site-packages/statsmodels/__init__.py'>
```

Module import paths

When you imported numpy and allensdk above, you didn't need to specify the location of the module files. How did Python know where to find them?

Python keeps a list of locations on your filesystem where installed packages may reside. When we ask Python to import a new module, it checks each location in the list for either a module or a package with the name you specified.

We can access this list from the `sys` module, which is part of the Python standard library:

In [13]:
```
import sys
sys.path
```

Out[13]:
```
['/home/luke/docs/swdb/python_bootcamp/code/solutions',
 '/home/luke/miniconda3/envs/swdb/lib/python312.zip',
 '/home/luke/miniconda3/envs/swdb/lib/python3.12',
 '/home/luke/miniconda3/envs/swdb/lib/python3.12/lib-dynload',
 '',
 '/home/luke/miniconda3/envs/swdb/lib/python3.12/site-packages']
```

When we install new packages, their files must be added to one of the entries in `sys.path`; otherwise, they will not be importable. (We could also modify `sys.path` to include the new location of the package, but this is generally discouraged.)

Let's see what happens when we try to import a module that cannot be found:

In [14]:
```
import not_a_real_module
# raises an exception
```

```
---------------------------------------------------------------------------
ModuleNotFoundError                       Traceback (most recent call last)
Cell In[14], line 1
----> 1 import not_a_real_module

ModuleNotFoundError: No module named 'not_a_real_module'
```

The error above is a common source of frustration for Python beginners, and it happens most frequently when there are multiple Python environments installed on a machine. When we install new packages, it is important to be aware of *which environment* the package is being installed into. Find the actual installed location of the module files, and verify that these are indeed in your `sys.path`.

Exercise 3.1:

Find the location of the statsmodels module on your disk and verify that it is in `sys.path`.

Using the OS module for dealing with paths, filenames and other miscellaneous operating system functions

Directories (folders) and files on your hard drive are organized in a tree structure. Folders contain subfolders, which contain more subfolders, and so on until the folder that contains your file of interest. The _path_ through these folders to a file can be represented as strings like this in Linux and Mac:

```
path = "/folder_a/folder_b/file.name"
```

And like this on Windows:

```
path = "C:\\folder_a\\folder_b\\file.name"
```

We use these path strings when instructing Python to read and write files.

Dealing with paths will be substantially easier if you first import the `os` module:

In [15]:
```
import os
```

Exercise 3.2:

`os.path.join` is a simple function that joins together multiple folder names using either a forward slash `/` or backslash `\` depending on the operating system.

```
os.path.join('Users', 'my_username', 'my_data_folder')
```

This is a convenient way to generate path names in code that you want to be portable.

Replace the names above with real folder names to a path on your system and see what is output:

In [16]:
```
name_1 = 'users'
name_2 = 'me'
name_3 = 'desktop'
print(os.path.join(name_1, name_2, name_3))
```

```
users/me/desktop
```

Another couple of handy functions are `getcwd` and `listdir`

In [17]:
```
print("current directory:")
print(os.getcwd())
print('\n')
print("files in current directory:")
os.listdir(os.getcwd())
```

```
current directory:
/home/luke/docs/swdb/python_bootcamp/code/solutions


files in current directory:
```

Out[17]:
```
['01_Basic_Python_I_Object_and_Data_Structures_solutions.html',
 '04_Introduction_To_Numpy_solutions.ipynb',
 '03_Modules_and_Packages_solutions.html',
 '__pycache__',
 '06_Introduction_to_Matplotlib_solutions.html',
 '01_Basic_Python_I_Object_and_Data_Structures_solutions.ipynb',
 '.ipynb_checkpoints',
 '02_Basic_Python_II_Control_Flow_and_Functions_solutions.html',
 'ephys_data.npy',
 'my_module.py',
 '08_Scikit-Learn_solutions.html',
 'my_package',
 '06_Introduction_to_Matplotlib_solutions.ipynb',
 'html',
 '05_Custom_Modules_solutions.ipynb',
 'my_data.npy',
 '03_Modules_and_Packages_solutions.ipynb',
 '04_Introduction_To_Numpy_solutions.html',
 '05_Custom_Modules_solutions.html',
 '02_Basic_Python_II_Control_Flow_and_Functions_solutions.ipynb',
 '07_Introduction_To_Pandas_solutions.html',
 '07_Introduction_To_Pandas_solutions.ipynb',
 '08_Scikit-Learn_solutions.ipynb']
```

> To find out if a path actually exists on your drive, use:
>
> `os.path.exists(path)`

Exercise 3.3:

Create a folder called `TempFolder` somewhere on your local drive

1. Generate the full path using the `os.path.join` function
2. Use the `os.path.exists` function to ensure that the folder doesn't already exist (confirm that it returns False)
3. Create the directory using `os.mkdir`
4. Confirm that it now exists using the `os.path.exists` function again
5. Navigate to the folder using your file explorer to confirm that it's there

In [18]:
```
new_path = os.path.join(os.getcwd(), 'TempFolder')

print(f"new path: {new_path}")
print(f"  exists? {os.path.exists(new_path)}")

if not os.path.exists(new_path):
    print("  creating directory")
    os.mkdir(new_path)
    print(f"  exists now? {os.path.exists(new_path)}")

# clean up
os.removedirs(new_path)
```

```
new path: /home/luke/docs/swdb/python_bootcamp/code/solutions/TempFolder
  exists? False
  creating directory
  exists now? True
```

A Brief Introduction to Scipy

The Scipy library has a lot of useful functions for signal processing, linear algebra, statistical testing, etc. Given its breadth, and its overlap with Numpy, we've elected not to cover it in detail here.

The Scipy library has ~20 subpackages

each subpackage contains a lot of useful functions

1. **Clustering package** (scipy.cluster)
2. Constants (scipy.constants)
3. **Discrete Fourier transforms** (scipy.fftpack)
4. Integration and ODEs (scipy.integrate)
5. Interpolation (scipy.interpolate)
6. Input and output (scipy.io)
7. **Linear algebra** (scipy.linalg)
8. Miscellaneous routines (scipy.misc)
9. **Multi-dimensional image processing** (scipy.ndimage)
10. Orthogonal distance regression (scipy.odr)
11. **Optimization and root finding** (scipy.optimize)
12. **Signal processing** (scipy.signal)
13. Sparse matrices (scipy.sparse)
14. Sparse linear algebra (scipy.sparse.linalg)
15. Compressed Sparse Graph Routines (scipy.sparse.csgraph)
16. Spatial algorithms and data structures (scipy.spatial)
17. Special functions (scipy.special)
18. **Statistical functions** (scipy.stats)

19. Statistical functions for masked arrays (scipy.stats.mstats)
20. C/C++ integration (scipy.weave)

Exercise 3.4:

Spend a few minutes exploring the scipy documnention here:

scipy reference